

Computational Frameworks

Streaming (Part 3)

OUTLINE

① Introduction to the streaming model (Part 1)

- Warm-up: finding the majority element

② Sampling (Part 2)

- Uniform sampling with reservoir sampling
- Finding frequent items with sticky sampling

③ Sketching (Part 3)

- Counting distinct elements with probabilistic counting
- Estimating moments with AMS sketch

Sketching

Sketches

Sketch: space-efficient data structure that can be used to provide (typically probabilistic) estimates of (statistical) characteristics of a data stream.

Frequency Moments

Consider a stream $\Sigma = x_1, x_2, \dots, x_n$, whose elements belong to a universe U .

For each $u \in U$ occurring in Σ define its (absolute) frequency

$$f_u = |\{j : x_j = u, 1 \leq j \leq n\}|,$$

i.e., the number of occurrences of u in Σ

Definition: Frequency Moments

For every $k \geq 0$, the k -th frequency moment F_k of Σ is

$$F_k = \sum_{u \in U} f_u^k$$

Frequency Moments

Relevant informations from frequency moments:

- $F_0 = \text{number of distinct items in } \Sigma$ (letting $0^0 = 1$). E.g., applications in web-log analysis.
- $F_1 = |\Sigma|$: trivial. *Easy to maintain with a counter*
- $1 - F_2/|\Sigma|^2 = \text{Gini index of } \Sigma$. It provides info on data skew: the closer to 0 and the higher is the skew. It is used for decision tree induction.

Streaming algorithm for computing F_0

Observation: $|F_0| \leq |U|$.

Exact computation: use $|U|$ counters.

Approximate computation: through Probabilistic Counting ([Flajolet, Martin IEEE FOCS 1983]), using a hash function h and $O(\log |U|)$ bits. The idea is the following:

- Map each $u \in U$ to a random integer $h(u) \in [0, |U| - 1]$.
 $\Rightarrow h(u)$ is a $O(\log |U|)$ -bit binary string.
- The more distinct elements in Σ , the more likely to have a $u \in \Sigma$ mapped to a string with many trailing 0's.

Probabilistic counting algorithm for estimating F_0

- Array C of $\lceil \log(|U|) + 1 \rceil$ bits, all initialized to zero.
- Hash Function $h : U \rightarrow [0..|U| - 1]$.

Hp: hash codes are independent random variables with uniform distribution.

- Function $t(i)$, for $i \in [0..|U| - 1]$: number of trailing zeroes in binary representation of i . ($i = 12 = (1100)_2 \rightarrow t(i) = 2$)

Algorithm:

- For each $x_i \in \Sigma$, set $C[t(h(x_i))] = 1$.
- After processing x_n , estimate F_0 as

$$\tilde{F}_0 = 2^R,$$

* $h(x_i) \equiv$ random integer in $[0, |U| - 1]$
* $t(h(x_i)) \equiv$ # of trailing 0's in $h(x_i)$

where R is the largest index of C with $C[R] = 1$.

Example

$$|U| = 26 \Rightarrow 5 \text{ bits}$$

$\Sigma = A, D, A, A, C, B, F, F, B, A, E, C$

x	$h(x)$	$t(h(x))$	C						
			0 1 2 3 4 5						
A	01001	0	<table><tr><td>1</td><td></td><td></td><td></td><td></td><td></td></tr></table>	1					
1									
D	11100	2	<table><tr><td>1</td><td></td><td>1</td><td></td><td></td><td></td></tr></table>	1		1			
1		1							
C	11000	3	<table><tr><td>1</td><td></td><td>1</td><td>1</td><td></td><td></td></tr></table>	1		1	1		
1		1	1						
B	01110	1	<table><tr><td>1</td><td>1</td><td>1</td><td>1</td><td></td><td></td></tr></table>	1	1	1	1		
1	1	1	1						
F	10100	2	<table><tr><td>1</td><td>1</td><td>1</td><td>1</td><td></td><td></td></tr></table>	1	1	1	1		
1	1	1	1						
E	10011	0	<table><tr><td>1</td><td>1</td><td>1</td><td>1</td><td></td><td></td></tr></table>	1	1	1	1		
1	1	1	1						

$$R = 3 \Rightarrow \tilde{F}_0 = 2^3 = 8 \quad F_0 = 6$$

Analysis: intuition

For simplicity, assume $|U|$ a power of 2. Let $x \in \Sigma$.

What is the probability that $h(x)$ has at least j trailing zeros?

- $\text{Prob}(t(h(x)) \geq 1) = 1/2$
- $\text{Prob}(t(h(x)) \geq 2) = 1/4$
- $\text{Prob}(t(h(x)) \geq 3) = 1/8$
- $\text{Prob}(t(h(x)) \geq j) = 1/2^j$

Hence: In order to set a bit of index $\geq j$ to 1, we must process, $\Omega(2^j)$ distinct elements in expectation! This explains why 2^R is a good estimate for F_0 .

Probabilistic guarantees

The following theorem summarizes the properties of the algorithm. We skip the proof.

Theorem

For a stream Σ of n elements, the **probabilistic counting algorithm** returns a value $\tilde{F}_0$ such that, for any $c > 2$:

$$\Pr(\tilde{F}_0 < F_0/c) \leq 1/c \quad \text{and} \quad \Pr(\tilde{F}_0 > cF_0) \leq 1/c,$$

hence

$$\Pr(F_0/c \leq \tilde{F}_0 \leq cF_0) \geq 1 - 2/c.$$

The algorithm requires a working memory of $O(\log |U|)$ bits, 1 pass, and $O(\log |U|)$ time per element.

Exercises

Exercise: High Probability Guarantees (median trick)

Consider a stream Σ with elements from a universe U and let F_0 be the number of distinct elements in Σ . Suppose that you run ℓ *independent instances* of the probabilistic counting argument, obtaining ℓ estimates $\tilde{F}_0^{(j)}$ for F_0 , with $1 \leq j \leq \ell$. Assume ℓ odd and let $\tilde{F}_0$ be the median value of the $\tilde{F}_0^{(j)}$'s. Show that with a suitable choice of $\ell = \Theta(\log |U|)$ we have:

$$\Pr(\tilde{F}_0 < F_0/16) \leq 1/|U| \quad \text{and} \quad \Pr(\tilde{F}_0 > 16F_0) \leq 1/|U|.$$

Hint: Use the theorem and the Chernoff bound.

To recap

- Streaming algorithms
 - Input received as a stream;
 - Small working memory and update time.
- Sampling
 - Reservoir sampling: to compute an uniform sample of a stream;
 - Sticky sampling: to find the frequent items in a stream.
- Sketching
 - A small summary of the data for estimating some characteristics of the stream (e.g., moments);
 - Probabilistic counter for counting the number of distinct elements.

References

- **[LRU14]** J. Leskovec, A. Rajaraman and J. Ullman. **Mining Massive Datasets**. Cambridge University Press, 2014. Chapter 4 (Section 4.1., 4.2, 4.4) (pdf provided in Moodle)
- **[DF08]** C. Demetrescu and I. Finocchi. **Algorithms for Data Streams**. In *Handbook of Applied Algorithms*, Chapter 8, Section 3. Wiley-IEEE Press, 2008. (pdf provided in Moodle)